Computer System Architecture

Lecture 2 - Digital Logic Review

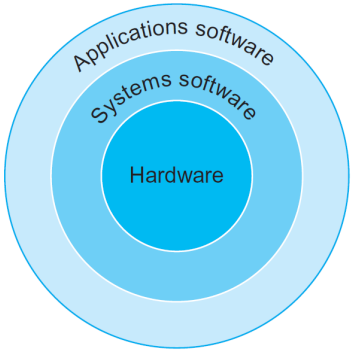
September 2026

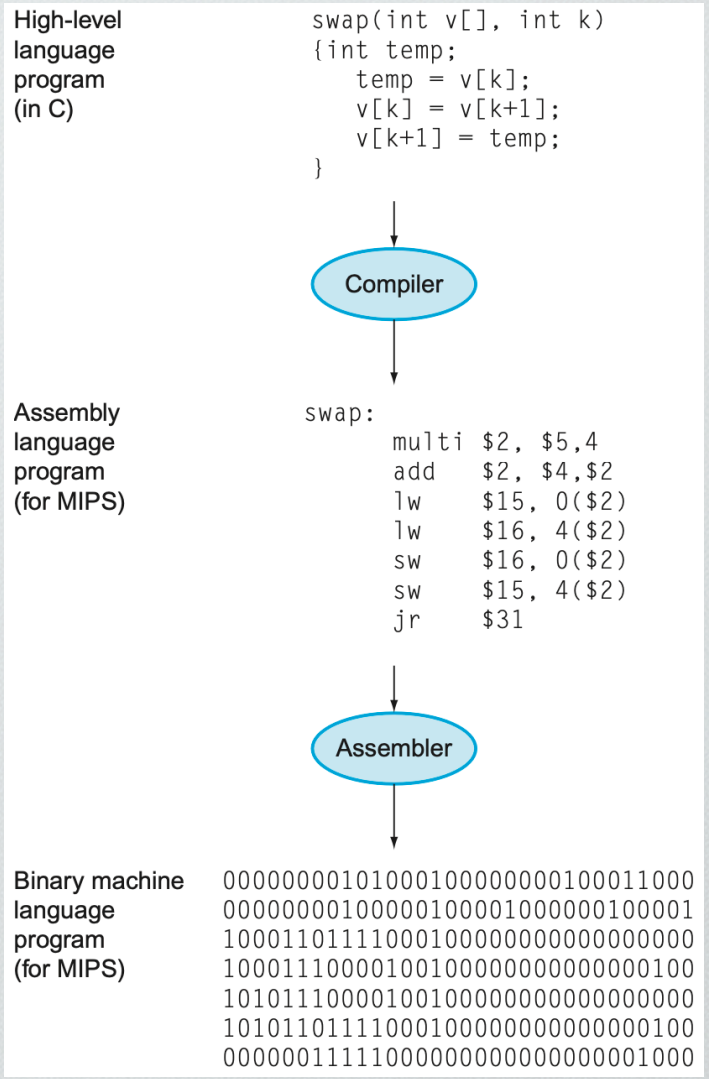
Review

System Software

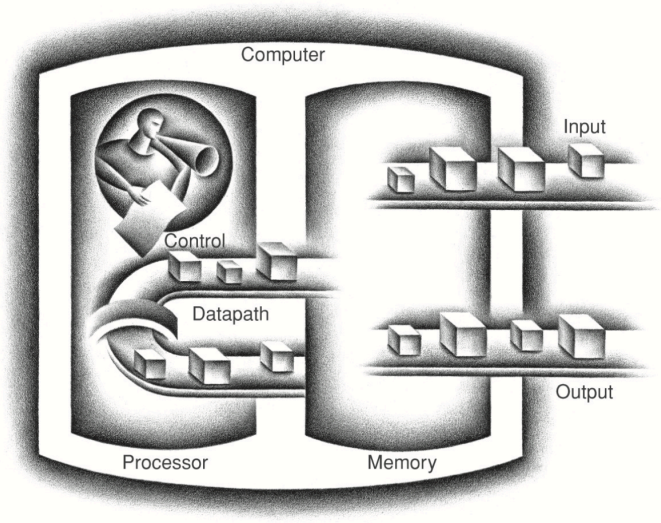
Compiler translates high-level language statements into assembly language statements.

Operating System (OS) manages hardware resources, memory, I/O operations, and task scheduling for applications.





Review



5 classic components of a computer.

01. Input

Bring information into the computer (e.g., keyboard).

02. Output

Send computation result out of the computer (e.g., display).

03. Memory

Store programs when they are running and data needed by them.

04. Datapath

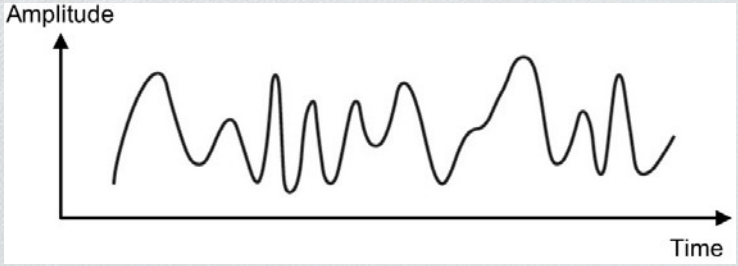
Component of the processor. Perform arithmetic and logical operations on data (e.g., ALU).

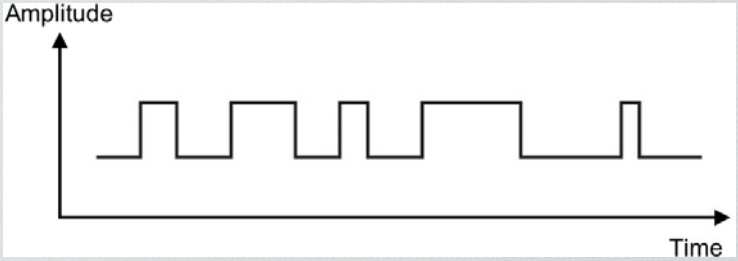
05. Control

Component of the processor. Command the datapath, memory, and I/O devices according to the instructions of the program.

Analog vs. Digital

- Analog Signal
  - Vary smoothly over time
  - Continuously valued
- Digital Signal
  - Maintain a constant level then changes to another constant level
  - Discretely valued





Base

- Base (Radix): the total number of digits allowed in the number system.

System Name	Decimal	Binary	Octal	Hexadecimal
Radix	10	2	8	16
	0	0000	0	0
	1	0001	1	1
	2	0010	2	2
	3	0011	3	3
	4	0100	4	4
	5	0101	5	5
	6	0110	6	6
	7	0111	7	7
	8	1000	10	8
	9	1001	11	9
	10	1010	12	A
	11	1011	13	B
	12	1100	14	C
	13	1101	15	D
	14	1110	16	E
	15	1111	17	F
	16	10000	20	10

Number Systems

- **Decimal numbers**

1's column
10's column
100's column
1000's column

$$5374_{10} = 5\times10^3 + 3\times10^2 + 7\times10^1 + 4\times10^0$$

five thousands | three hundreds | seven tens | four ones

- **Binary numbers**

1's column
2's column
4's column
8's column

$$1101_2 = 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 = 13_{10}$$

one eight | one four | zero two | one one

Decimal vs Binary Numbers

- Binary to decimal conversion:
  - Convert 10011_2 to decimal
  - $16\times1 + 8\times0 + 4\times0 + 2\times1 + 1\times1 = 19_{10}$
- Decimal to binary conversion:
  - Convert 47_{10} to binary
  - $32\times1 + 16\times0 + 8\times1 + 4\times1 + 2\times1 + 1\times1 = 101111_2$
- *N*-digit decimal number
  - How many values? 10^N Range: $[0, 10^N-1]$
  - Example: 3-digit decimal number:
  - $10^3 = 1000$ possible values Range: [000, 999]
- *N*-digit binary number
  - How many values? 2^N Range: $[0, 2^N-1]$
  - Example: 3-digit binary number:
  - $2^3 = 8$ possible values Range: $[000_2, 111_2]$

Hexadecimal Numbers

- Hexadecimal to binary conversion:
  - $4AF_{16}$ (or 0x4AF)
  - $0100\ 1010\ 1111_2$

- Hexadecimal to decimal conversion:
  - $4AF_{16}$ (or 0x4AF)
  - $16^2 \times 4 + 16^1 \times 10 + 16^0 \times 15$
    $= 1199_{10}$

Unsigned and Signed Numbers

Hexadecimal	Binary	Decimal (Unsigned)	Decimal (2's Complement)
0x8F	0111 1111	127	127
0x8E	0111 1110	126	126
…	…	…	…
0x02	0000 0010	2	2
0x01	0000 0001	1	1
0x00	0000 0000	0	0
0xFF	1111 1111	$2^8-1 = 255$	−1
0xFE	1111 1110	254	−2
…	…	…	…
0x82	1000 0010	130	−126
0x81	1000 0001	129	−127
0x80	1000 0000	128	−128

Complement all bits

1000 0000

Then add 1

1000 0001

Complement all bits

0000 0001

Then add 1

0000 0010

Exercise

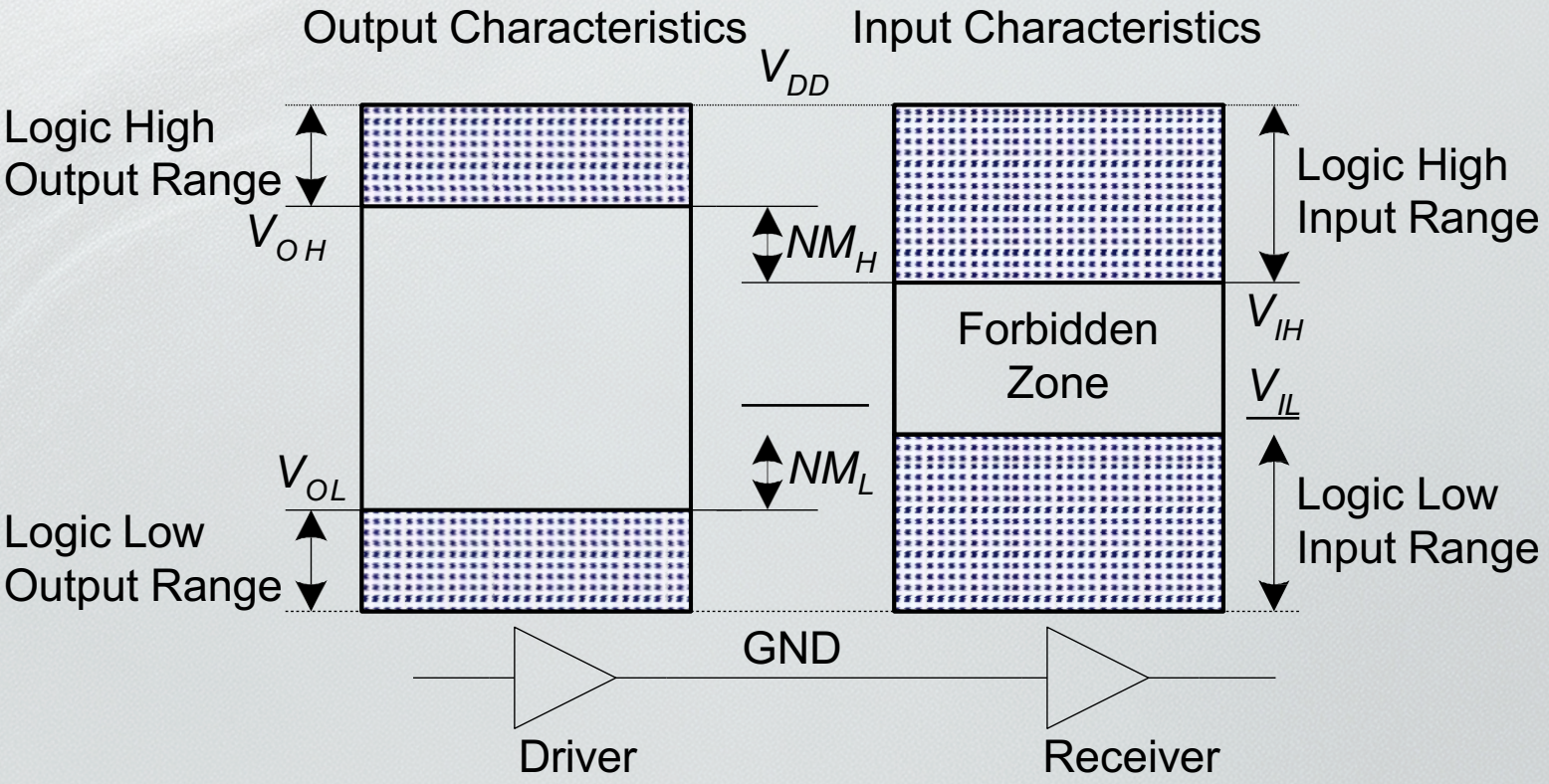
- **Ex 1:** For a *n*-bit 2's complement signed binary numeral system, what's the largest positive number and the smallest negative number?

Digital Circuits

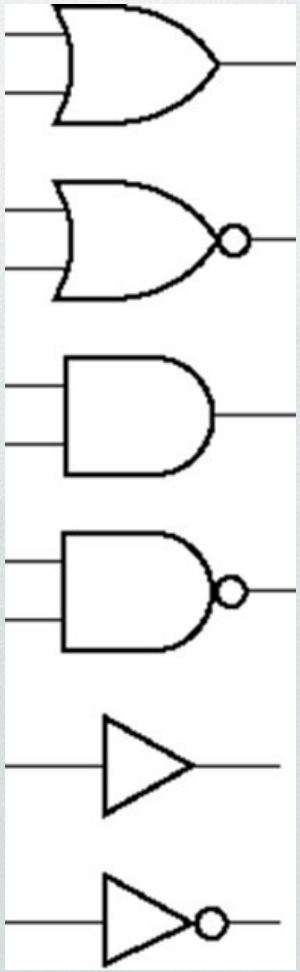
- **Combinational** circuits consist of logic gates with inputs and outputs. The output is directly dependent on the input at the current state.
- **Sequential** circuits have memory. The output depends on both the current inputs as well as the stored values of the circuit.

Logic Levels

- **Active High**: High voltage activates/enables the pin
- **Active Low**: Low voltage activates/enables the pin



Logic Gates



OR

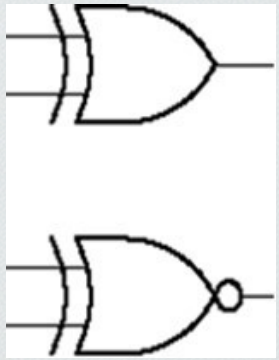
NOR

AND

NAND

buffer

NOT (inverter)



XOR

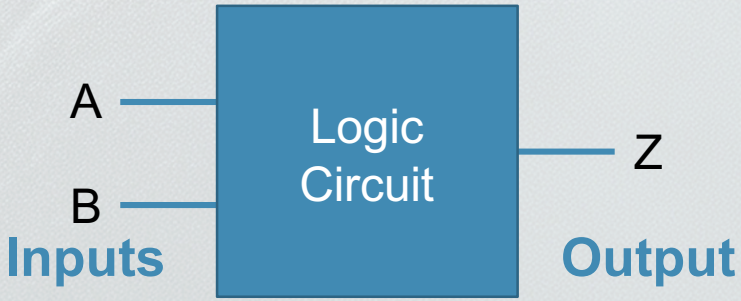
XNOR



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

Truth Table

- A tool for describing how the output of a logic circuit depends on the logic levels presented at the inputs.
- The number of input combinations will equal 2^N for an *N*-input truth table

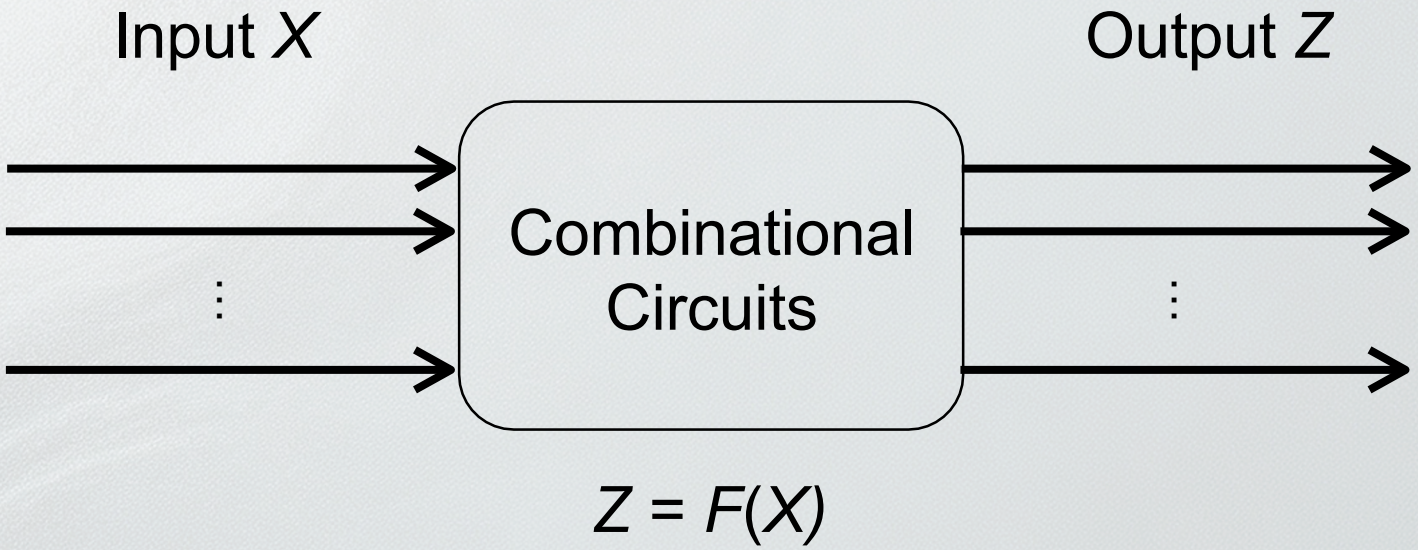


A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

Exercise

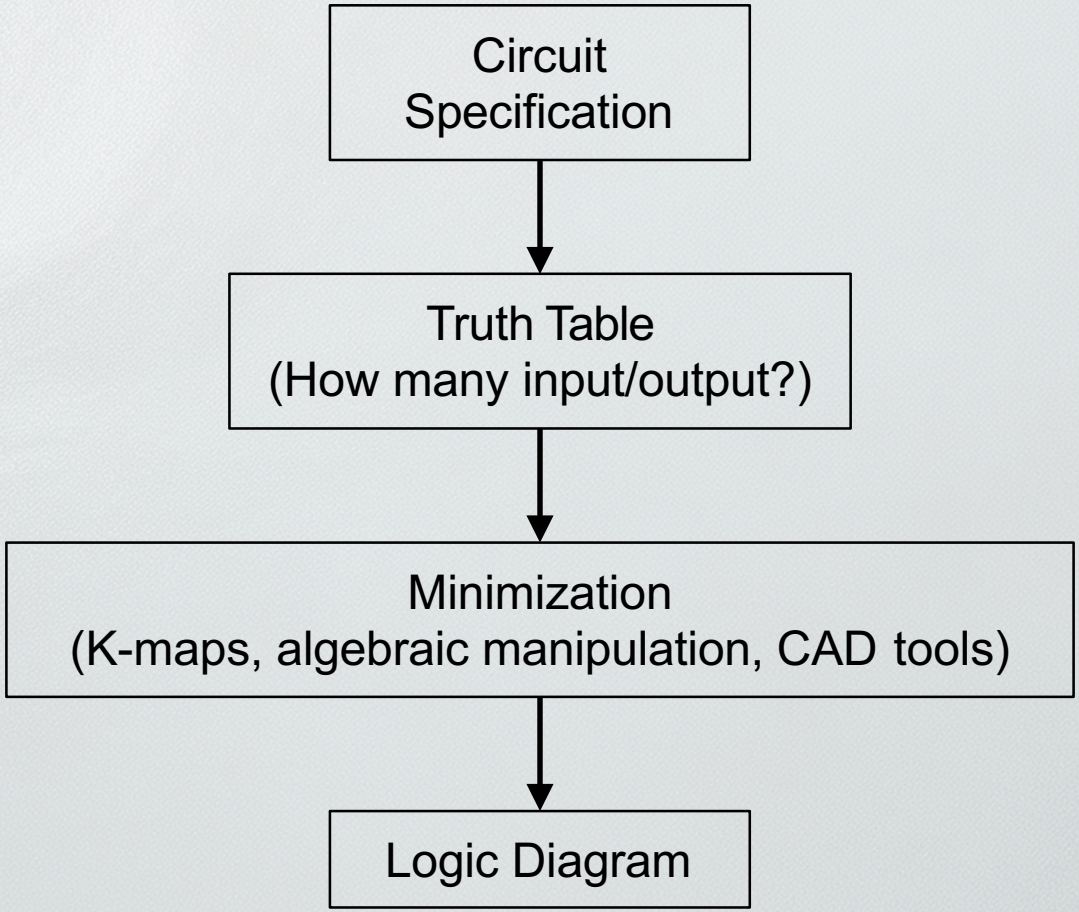
- **Ex 2:** Draw the truth table of a 3-input XNOR gate.

Combinational Circuits



- In combinational circuits, the output at any time is a direct function of the applied external inputs

Design Procedure of Combinational Circuits

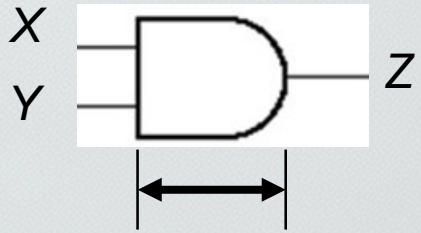


Exercise

- **Ex 3:** Implement AB+CD using NAND gates only.

Propagation Delay

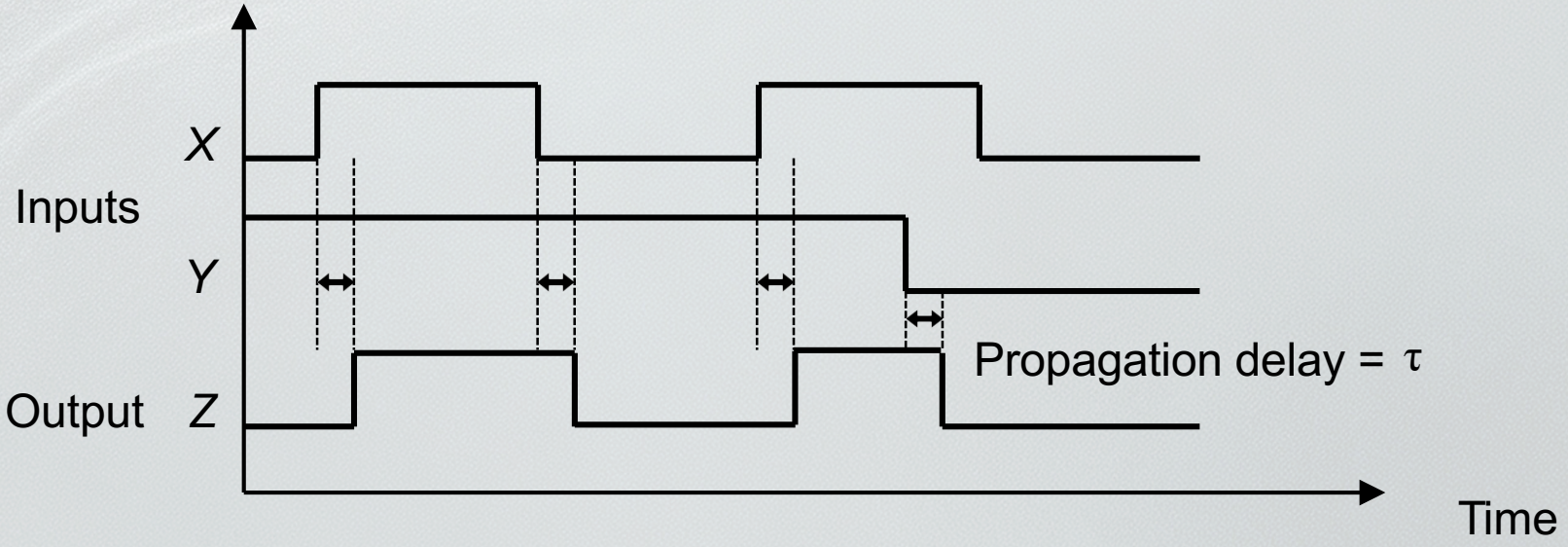
- Propagation delay: the delay between the arrival of signal at the input and the change of output.
- A circuit is considered to be fast if its propagation delay is small.



Delay between input (*X*, *Y*) and change in output *Z*.

Timing Diagram

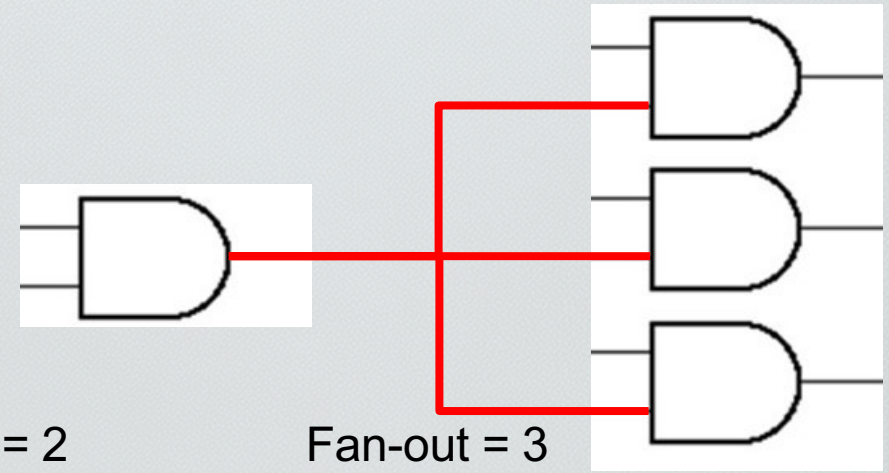
- The timing diagram shows the values of the input signals to a circuit with respect to time as a waveform, together with another waveform for the output.



Fan-in & Fan-out

- Fan-in is the number of inputs to a gate.

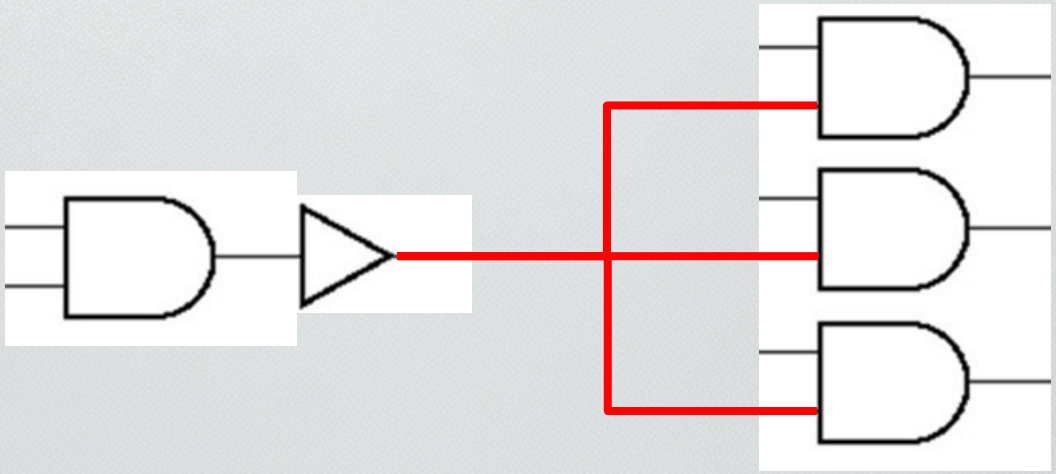
  E.g. fan-in = 3 for a 3-input XNOR gate.
- Fan-out is the number of load gates that can be driven by a driver gate.
- There is a limitation on fan-in and fan-out for any gate.



Fan-in = 2 Fan-out = 3

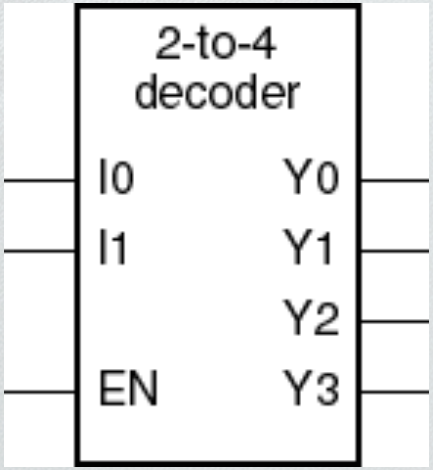
Buffers

- Output = Input
- Help increase the driving strength (how much load a gate can drive) of a circuit
- Do not affect the logical level



Decoders

- Information is represented by binary codes
- A decoder is a minterm generator for producing product that includes all input variables, e.g. $\bar{A}BC$, $A\bar{B}\bar{C}$, ABC
- *N* inputs, 2^N outputs (One-hot: only one output HIGH at once)

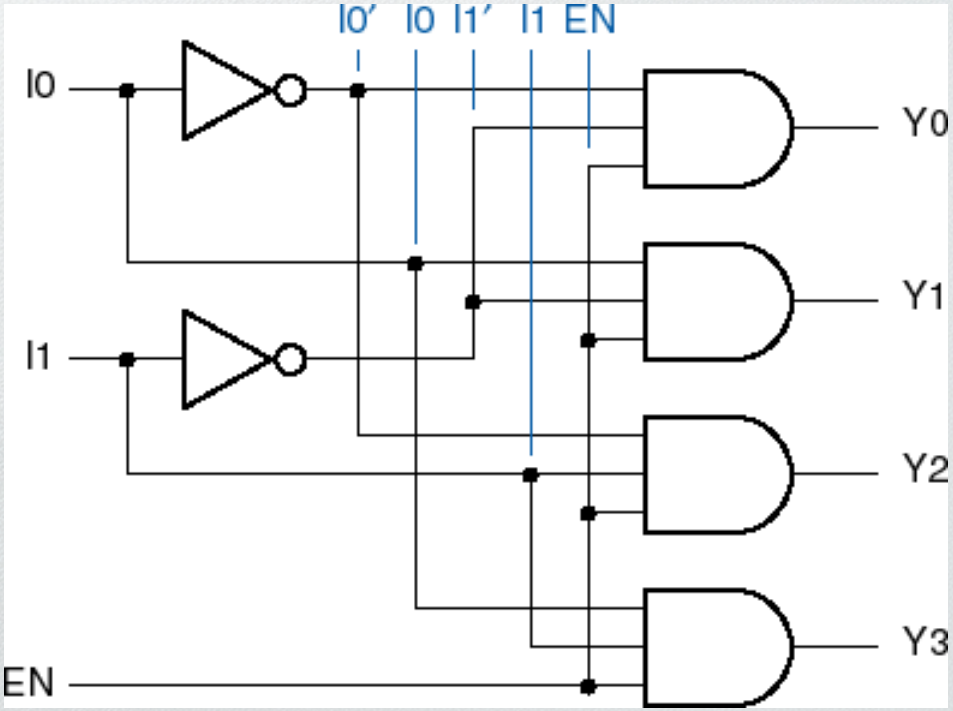


Binary 2-to-4 Decoder

Inputs			Outputs			
EN	I1	I0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Note: “x” denotes don’t care.

Decoders

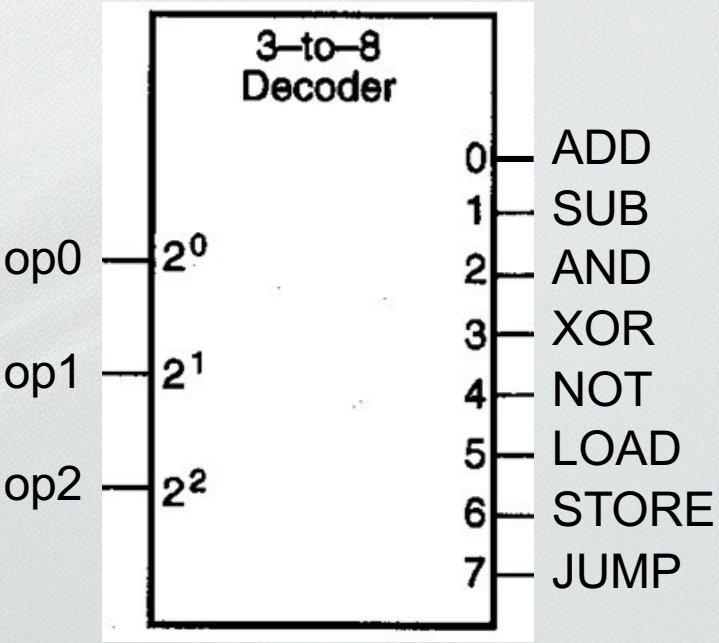


2-to-4 Decoder Logic Diagram

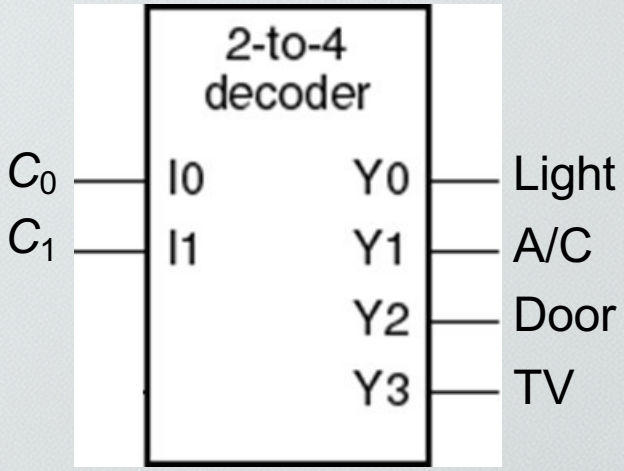
Inputs			*Outputs*			
EN	I1	I0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Decoder Applications

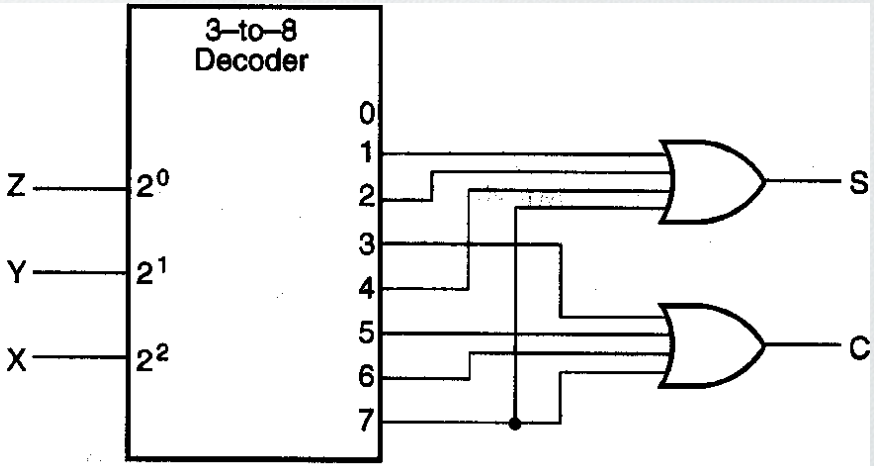
- Decode 3-bit opcodes:



- Home automation:



Decoders as Binary Adder



Inputs			Outputs							
X	Y	Z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	**1**	0	0	0	0	0	0	0
0	0	1	0	**1**	0	0	0	0	0	0
0	1	0	0	0	**1**	0	0	0	0	0
0	1	1	0	0	0	**1**	0	0	0	0
1	0	0	0	0	0	0	**1**	0	0	0
1	0	1	0	0	0	0	0	**1**	0	0
1	1	0	0	0	0	0	0	0	**1**	0
1	1	1	0	0	0	0	0	0	0	**1**

$$S = \bar{X}\bar{Y}Z + \bar{X}Y\bar{Z} + X\bar{Y}\bar{Z} + XYZ$$

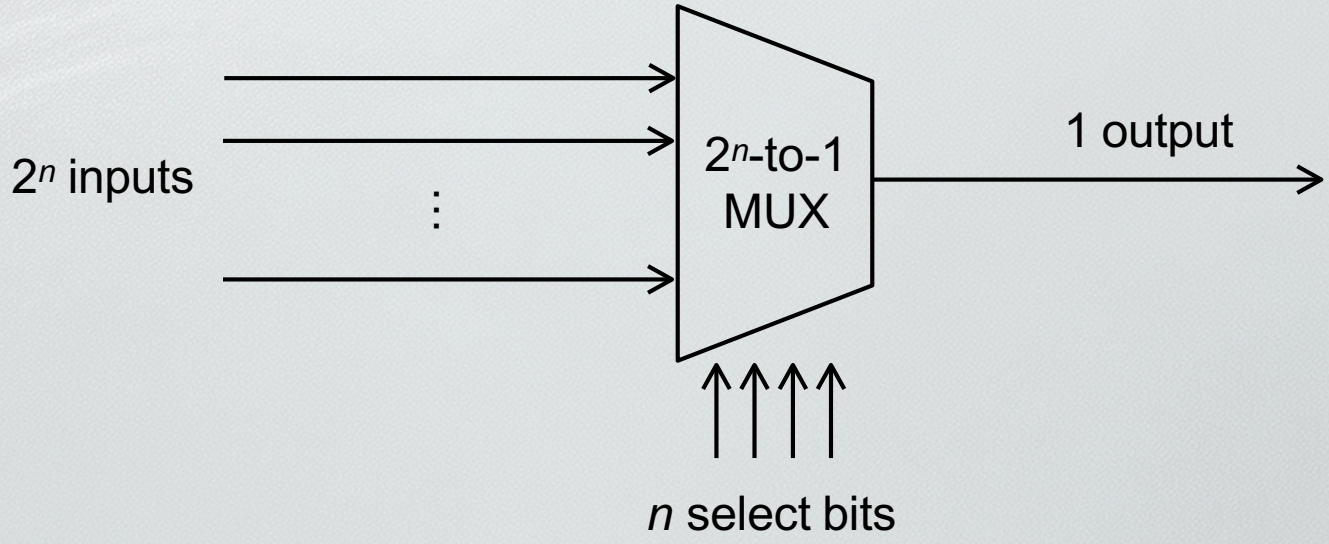
$$C = XY + XZ + YZ$$

TABLE 3-8
Truth Table of Full Adder

Inputs			Outputs	
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

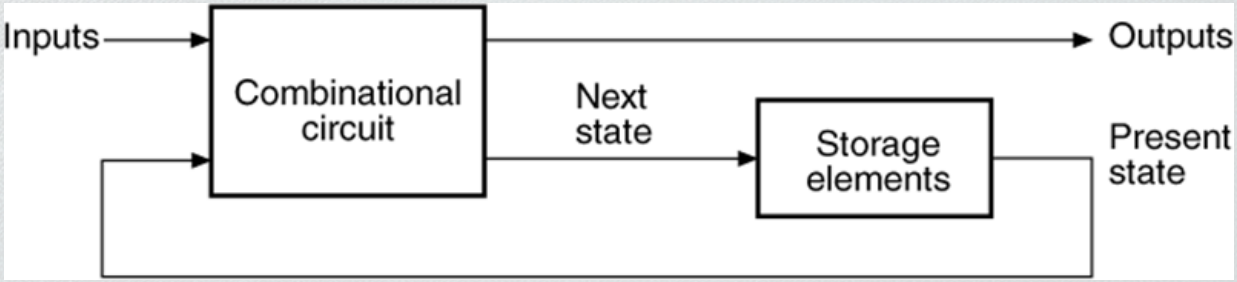
Multiplexers

- A data selector for directing one of 2^n input to the output.
- Selection is done based on a set of *n* select bits.



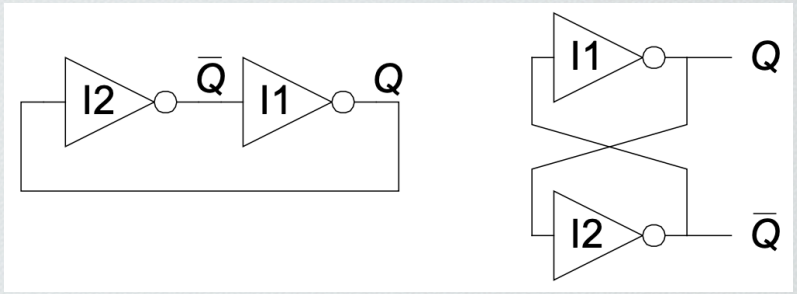
Sequential Circuits

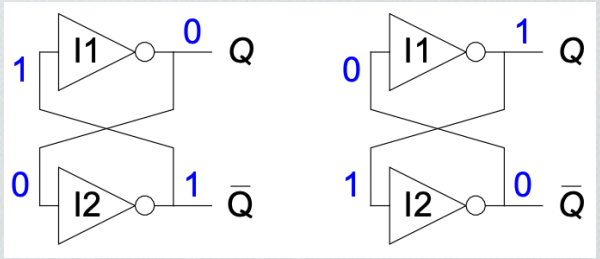
- Have memory (short-term)
- Storage elements (latch, flip-flop) store state



Bistable Circuit

- Fundamental building block of other state elements
  - Two outputs: Q, $\overline{Q}$
  - No inputs
- Consider the two possible cases:
  - $Q = 0$: then $\overline{Q} = 1$, $Q = 0$ (consistent)
  - $Q = 1$: then $\overline{Q} = 0$, $Q = 1$ (consistent)

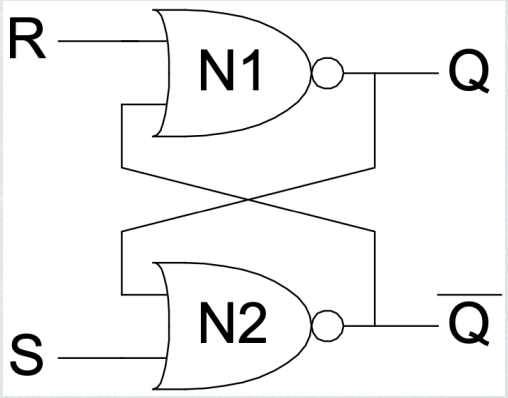




- Stores 1 bit of state in the state variable, Q (or $\overline{Q}$).
- Problem: no inputs to control the state.

Set/Reset Latch

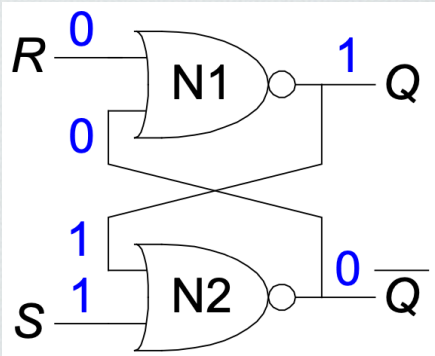
- SR Latch



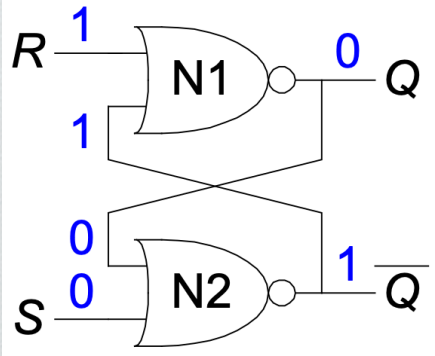
- Consider the four possible cases:
  - $S = 1, R = 0$
  - $S = 0, R = 1$
  - $S = 0, R = 0$
  - $S = 1, R = 1$

SR Latch Analysis

- $S = 1$, $R = 0$:
  - then $Q = 1$ and $\overline{Q} = 0$



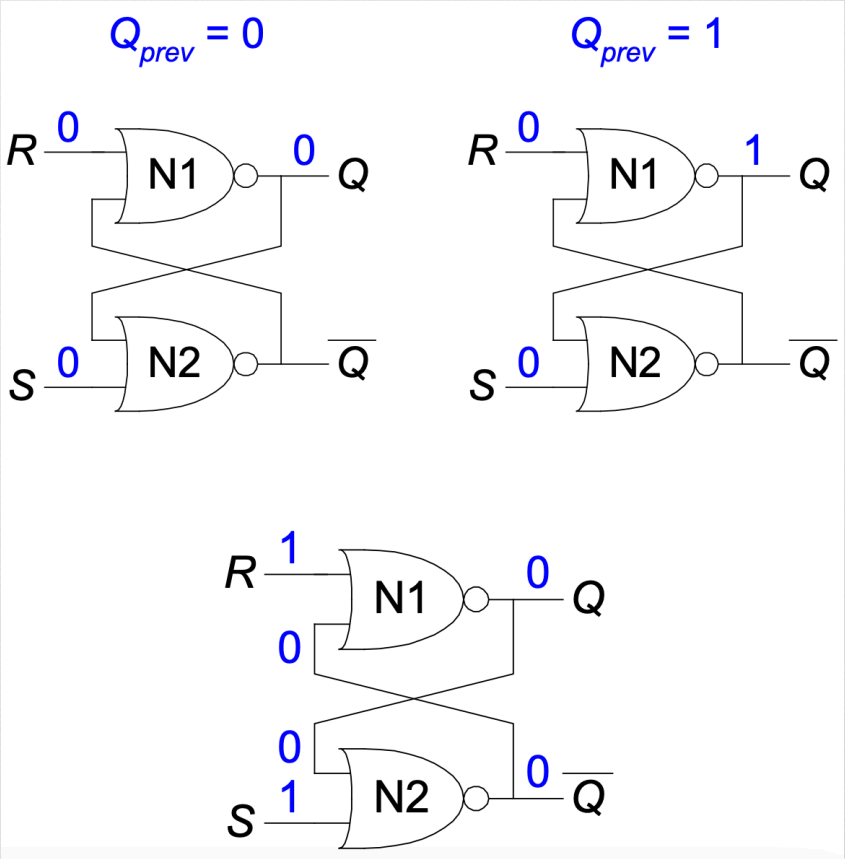
- $S = 0$, $R = 1$:
  - then $Q = 0$ and $\overline{Q} = 1$



SR Latch Analysis

- $S = 0, R = 0$:
  - then $Q = Q_{prev}$
  - Memory

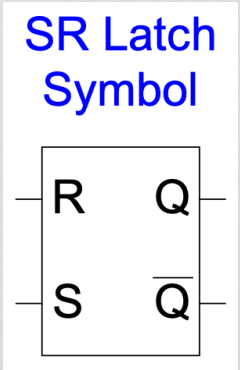
- $S = 1, R = 1$:
  - then $Q = 0, \overline{Q} = 0$
  - Invalid ($Q = \overline{Q}$)



SR Latch Symbol

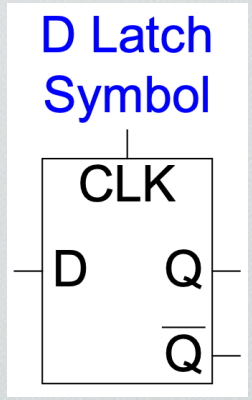
- SR (Set/Reset) Latch stores one bit of state (Q)
- Control stored value using *S*, *R* inputs
  - Set: Make the output 1 (S = 1, R = 0, Q = 1)
  - Reset: Make the output 0 (S = 0, R = 1, Q = 0)

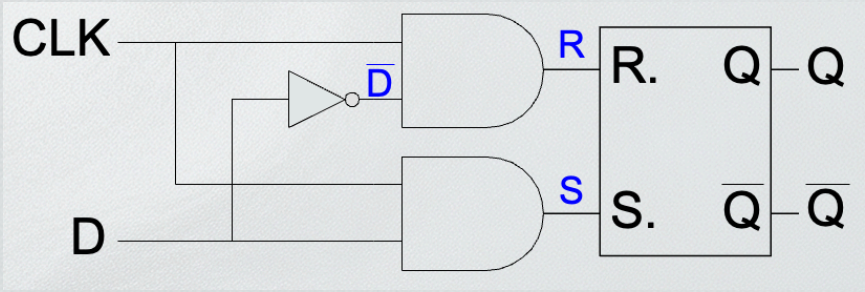
S	R	Q	$\overline{Q}$
0	0	last Q	last $\overline{Q}$
0	1	0	1
1	0	1	0
1	1	0	0



Gated D Latch

- Two inputs: *CLK*, *D*
  - *CLK*: controls when the output changes.
  - *D* (data input): controls what the output changes to.
  - When *CLK* = 1, transparent
    D passes through to *Q*
  - When *CLK* = 0, opaque
    Q holds its previous value

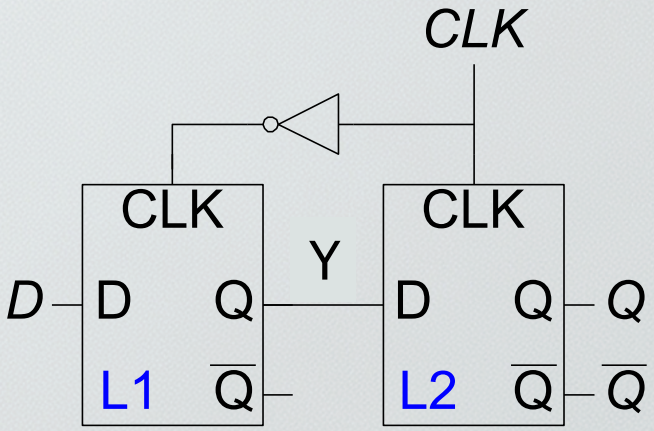




CLK	*D*	$\overline{D}$	*S*	*R*	*Q*	$\overline{Q}$
0	X	$\overline{X}$	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

Master-Slave D Flip-Flop

- Two back-to-back D latches controlled by complementary clocks.
- When CLK = 0
  - L1 is transparent, L2 is opaque
  - *D* passes through to *Y*
- When CLK = 1
  - L1 is opaque, L2 is transparent
  - *Y* passes through to *Q*

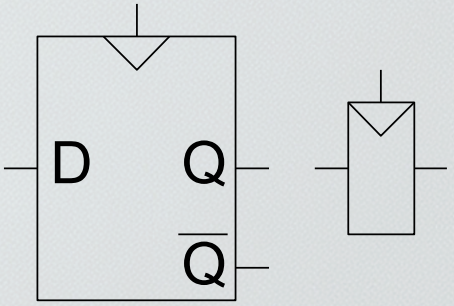


- On the rising edge of the clock (CLK rises from 0 to 1), D passes through to *Q*.

Edge-Triggered D Flip-Flop

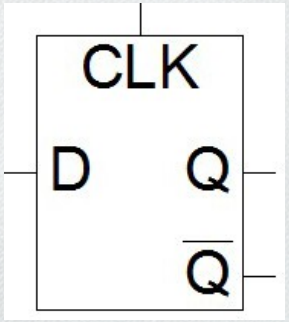
- Inputs: *CLK*, *D*
- Samples *D* on rising edge of *CLK*
  - When *CLK* rises from 0 to 1, *D* passes through to *Q*
  - Otherwise, *Q* holds its previous value
- Edge-triggered
  - Activated on the clock edge

D Flip-Flop Symbols

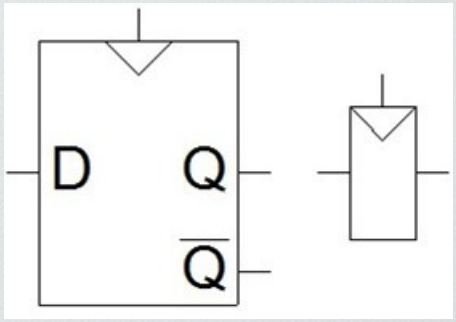


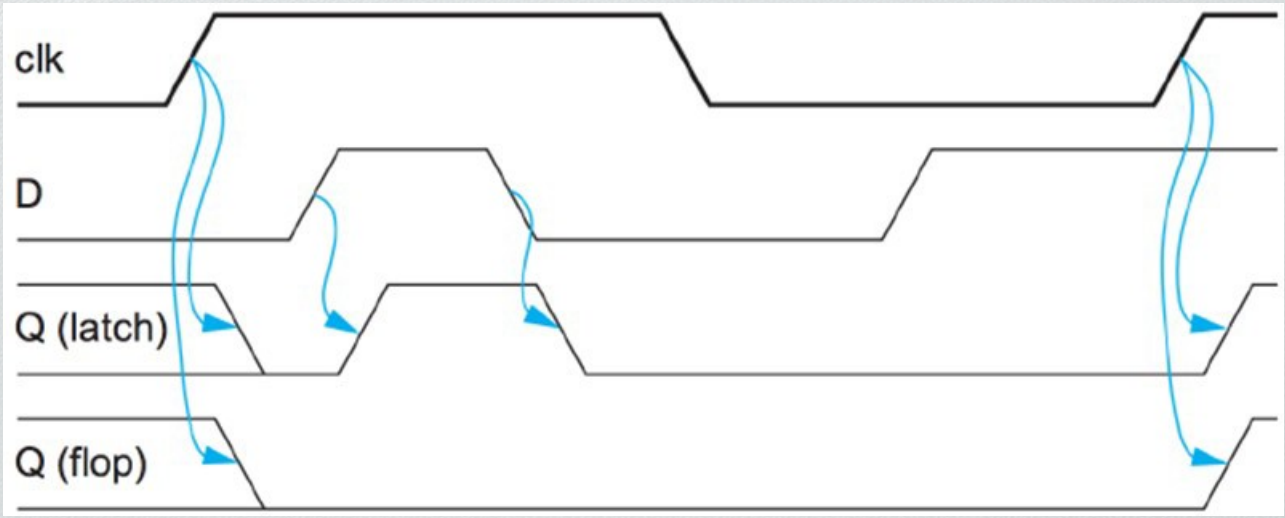
Latch and Flip-Flop

- Latch is level-sensitive.



- Flip-flop is edge-triggered.



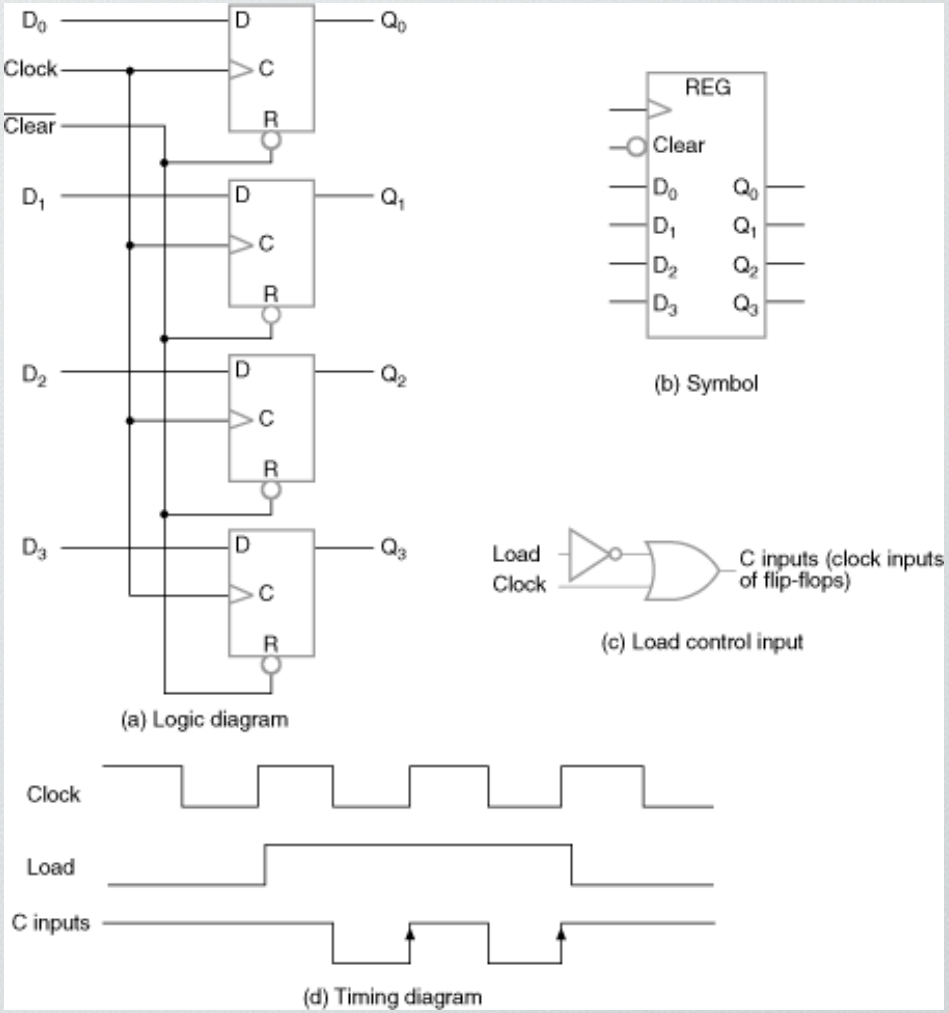


Registers

- A register is a simple memory device composed of a series of flip-flops wired together so that they share a common clock pulse.
- It stores data and instructions for quick processing.
- Registers come in various sizes and types. The size of a register is closely tied to the structure of the processor.

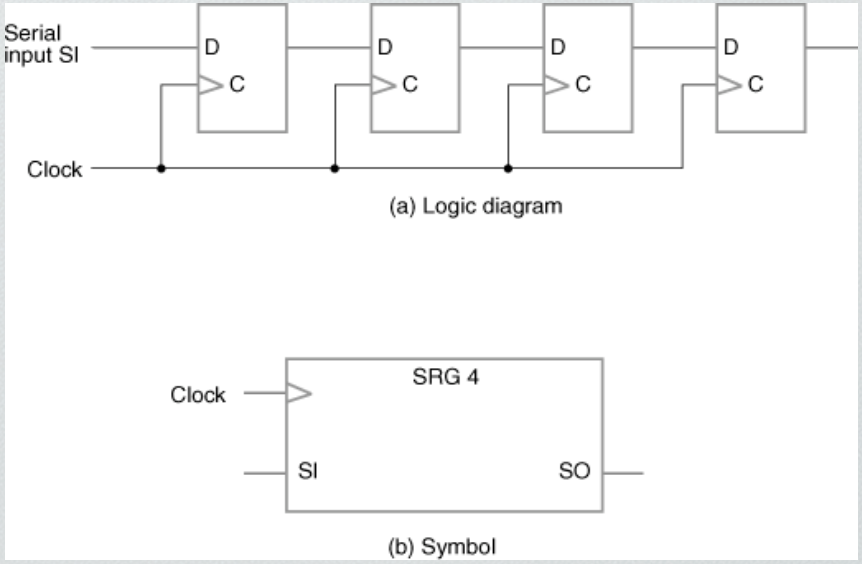
4-Bit Register

- 4 D flip-flops
- Common Clock
  - At each positive-edge, 4 bits are loaded in parallel, overwriting previous data.
- Common Clear
  - When Clear = 0, all flip-flops are cleared asynchronously

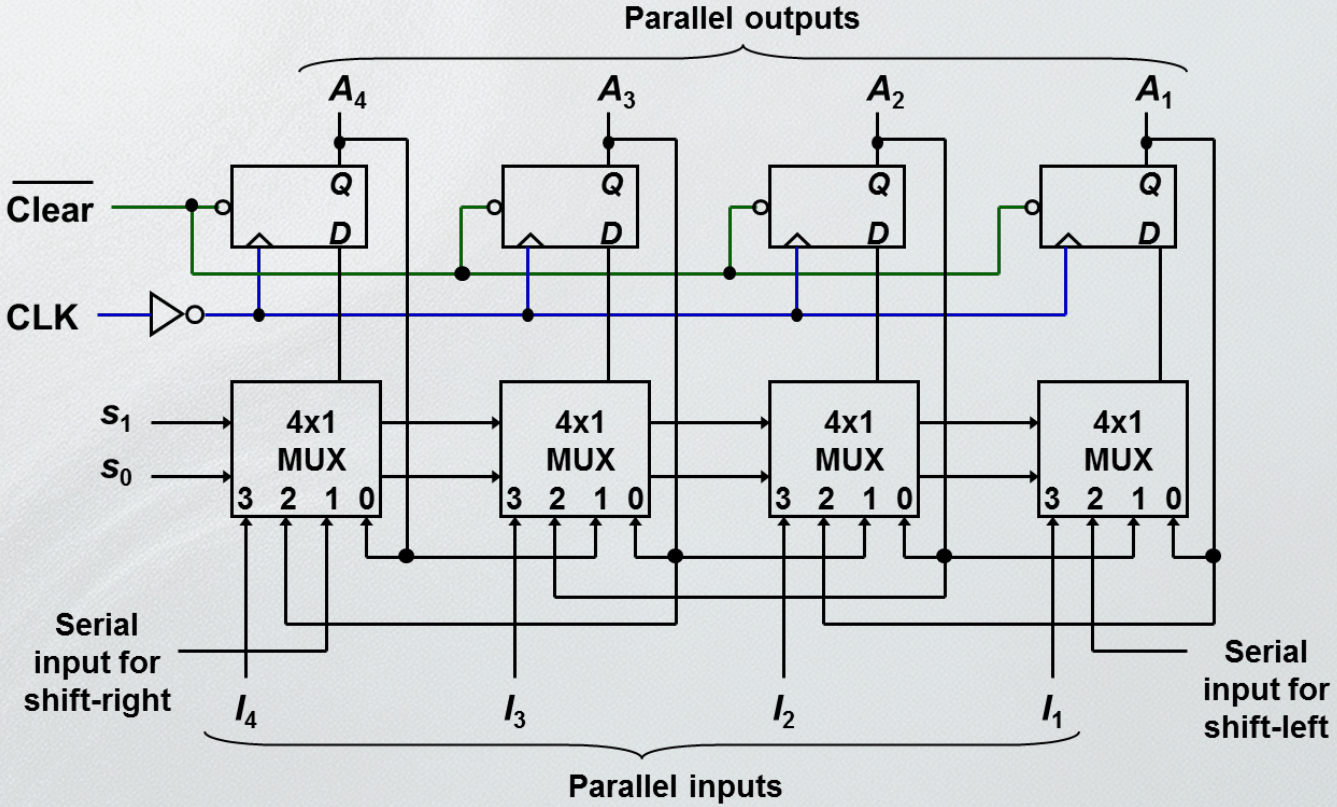


Shift Registers

- Shift register: A register which can shift its stored bits in one or both directions.
- Serial-in, serial-out
- At each positive-edge, 4 bits are loaded in parallel, overwriting previous data.



Bidirectional Shift Register with Parallel Load



Mode Control		Register Operation
s_1	s_0	
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load